```
class Area:
  def __init__(self,l,b):
    self.length=l
    self.breath=b

  def returnArea(self):
    area=l*b
    print(area)

length=int(input("Enter length:"))
breath=int(input("Enter breath:"))
obj=Area(length,breath)
obj.returnArea
```

```
Enter length:6
Enter breath:7
```

```
Area.returnArea
def returnArea()
```

```
<no docstring>
```

Abstraction

```
from abc import ABC, abstractmethod
class Test(ABC):
  @abstractmethod
  def m1(self):
    pass

  @abstractmethod
  def m2(self):
    pass

class MyPython(Test):
  pass

m=MyPython()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_604/564682575.py in <cell line: 0>()
     12   pass
     13
---> 14 m=MyPython()

TypeError: Can't instantiate abstract class MyPython without an implementation for abstract methods 'm1', 'm2'
```

```
from abc import ABC, abstractmethod
class Test(ABC):
  @abstractmethod
  def m1(self):
    pass

  @abstractmethod
  def m2(self):
    pass

class MyPython(Test):
  def m1(self):
    print("m1 implemented")
  def m2(self):
    print("m2 implemented")
m=MyPython()
m.m1()
m.m2()
```

```
m1 implemented
m2 implemented
```

```
class MyPython(Test):
  def m1(self):
    print("m1 implemented")
```

```
  @abstractmethod
  def m2(self):
    pass

class Child(MyPython):
      def m2(self):
       print("m2 implemented")

m=Child()
m.m1()
m.m2()
```

```
m1 implemented
m2 implemented
```

```
from abc import ABC, abstractmethod
class Test(ABC):
  def __init__(self):
    print("Constructor is called...")

@abstractmethod
def m1(self):
  pass

class MyPython(Test):
  def m1(self):
    print("m1 Method implementation")

m=MyPython()
m.m1()
```

```
Constructor is called...
m1 Method implementation
```

Incapsulation

```
class Test1:
  _b=None
  __c=None
  def __init__(self):
    self.a=101
    self._b=201
    self.__c=301

  def show1(self):
    print("a=",self.a)
    print("b=",self._b)
    print("c=",self.__c)

class Test2(Test1):
    def show2(self):
      print("a=",self.a)
      print("b=",self._b)
      #print("c=",self.__c)

class Test3(Test2):
    def show3(self):
      print("a=",self.a)
      print("b=",self._b)
      #print("c=",self.__c)

t1=Test1()
t1.show1()
t2=Test2()
t2.show2()
t3=Test3()
t3.show3()
```

```
a= 101
b= 201
c= 301
a= 101
b= 201
```

```
a= 101
b= 201
```

Lgical Codes

Find second largest no. in 7,3,9,2,8

```
list=[7,3,9,2,8]
list.sort()
print(list)
print(list[-2])
```

```
[2, 3, 7, 8, 9]
8
```

move zeo to end in 0,1,0,3,12

```
list1=[0,1,0,3,12]
for i in list1:
  if i==0:
    list1.remove(i)
    list1.append(i)
print(list1)
```

```
[1, 3, 12, 0, 0]
```

we have 1,2,3 and 2,3,4 and 3,4,5 find common element

```
list1=[1,2,3]
list2=[2,3,4]
list3=[3,4,5]
for i in list1:
  if i in list2 and i in list3:
    print(i)
```

```
3
```

find max consequtive number

```
A=[1,1,0,1,1,1,0,1,1,1,1]
count=0
for i in A:
  if A[i]==A[i+1]:
    count+=1

print(count)
```

```
2
```

```
a=[1,2,3,4,5,6,7,8,9]
a[::2]=10,20,30,40,50,60
print(a)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_604/635154727.py in <cell line: 0>()
      1 a=[1,2,3,4,5,6,7,8,9]
----> 2 a[::2]=10,20,30,40,50,60
      3 print(a)

ValueError: attempt to assign sequence of size 6 to extended slice of size 5
```

```
a=[1,2,3,4,5,6,7,8,9]
a[::2]=10,20,30,40,50
print(a)
```

```
[10, 2, 20, 4, 30, 6, 40, 8, 50]
```

```
a=[1,2,3,4,5]
print(a[3:0:-1])
print(a[3:0])
```

```
[4, 3, 2]
[]
```

```
def func(value,values):
  var=1
  values[0]=44

t=3
v=[1,2,3]
func(t,v)
print(t,v[0])
```

```
3 44
```

```
arr=[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]]
for i in range(4):
  print(arr[i].pop())
```

```
4
8
12
16
```

```
arr=[1,2,3,4,5,6]
for i in range(1,6):
  arr[i-1]=arr[i]
for i in range(0,6):
  print(arr[i],end=" ")
```

```
2 3 4 5 6 6
```

```
def f(i, values=[]):
  values.append(i)
  print(values)
  # return values
f(1)  # calling values
f(2)
f(3)
```

```
[1]
[1, 2]
[1, 2, 3]
```

```
fruit_list1=['Apple','Berry','Cherry','Papaya']
fruit_list2=fruit_list1
fruit_list3=fruit_list1[:]
fruit_list2[0]='Guava'
fruit_list3[1]='Kiwi'

sum=0
for ls in (fruit_list1, fruit_list2,fruit_list3):
  if ls[0]=='Guava':
    sum+=1
  if ls[1]=='Kiwi':
    sum+=20
print(sum)
```

```
22
```

```
data=[[[1,2],[3,4]],[[5,6],[7,8]]]
def fun(m):
  v=m[0][0]
  for row in m:
    for element in row:
      if v < element: v=element
  return v
print(fun(data[0]))
```

```
4
```

```
a={(1,2):1,(2,3):2,(4,5):3} #we can accept tuple as a key
print(a[4,5])
```

```
3
```

```
a={'a':1,'b':2,'c':3}
print(a['a','b'])         #key error
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_526/4244323278.py in <cell line: 0>()
      1 a={'a':1,'b':2,'c':3}
----> 2 print(a['a','b'])         #key error

KeyError: ('a', 'b')
```

```
fruit={}
def addone(index):
  if index in fruit:
    fruit[index]+=1
  else:
    fruit[index]=1
addone('Apple')
addone('Banana')
addone('apple')
print(len(fruit))
```

```
3
```

```
arr={}
arr[1]=1
arr['1']=2
arr[1]+=1
print(arr)
sum=0
for k in arr:
  sum+=arr[k]
print(sum)
```

```
{1: 2, '1': 2}
4
```

```
dict={}
dict[1]=1
dict['1']=2
dict[1.0]=4   # updating value of dict[1]
print(dict)
sum=0
for k in dict:
  sum+=dict[k]
print(sum)
```

```
{1: 4, '1': 2}
6
```

```
dict={}
dict[1]=1
dict['1']=2
dict[1.5]=4
print(dict)
sum=0
for k in dict:
  sum+=dict[k]
print(sum)
```

```
{1: 1, '1': 2, 1.5: 4}
7
```

```
dict={}
dict[(1,2,4)]=8
dict[(4,2,1)]=10
dict[(1,2)]=12
sum=0
for k in dict:
```

```
  sum+=dict[k]
print(sum)
print(dict)
```

```
30
{(1, 2, 4): 8, (4, 2, 1): 10, (1, 2): 12}
```

```
box={}
jars={}
crates={}
box['biscuits']=1
box['cake']=3
jars['jars']=4
crates['box']=box
crates['jars']=jars
print(len(crates[box]))   #typeerroe
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_526/3796025354.py in <cell line: 0>()
      7 crates['box']=box
      8 crates['jars']=jars
----> 9 print(len(crates[box]))   #typeerroe

TypeError: unhashable type: 'dict'
```

```
dict={'c':97,'a':96,'b':98}
for _ in sorted(dict):
  print(dict[_])
```

```
96
98
97
```

```
rec={"Name": "Python", "age":"20"}
r=rec.copy()
print(id(r))
print(id(r)==id(rec))
print(id(rec))
```

```
138922434446144
False
138922434458944
```

```
for i,j in zip(range(1,6),range(5,0,-1)):
  print(i," ",j)
```

```
1   5
2   4
3   3
4   2
5   1
```

```
for i,j in zip(range(1,6),range(5,0,-1)):
  if i==3 and j==3:
    continue
  print(i," ",j)
```

```
1   5
2   4
4   2
5   1
```

```
f=open("myfle.txt",'w')
f.write("Ashish\n")
f.write("Amar\n")
f.write("Akash\n")
print("Write operator done successfully")
```

```
Write operator done successfully
```

```
f=open("myfile.txt",'w')
l=["Neha\n","Alisha\n","Amar\n","Aditya\n","Ashish\n"]
f.writelines(l)
```

```
print("Write operation done successfully")
f.close()
```

```
Write operation done successfully
```

```
f=open("myfile.txt",'r')
data=f.read()
print(data)
f.close()
```

```
Neha
Alisha
Amar
Aditya
Ashish
```

```
f=open("myfile.txt",'r')
data=f.read(12)
print(data)
f.close()
```

```
Neha
Alisha
```

```
f=open("myfile.txt",'r')
l1=f.readlines()
print(l1)
for x in l1:
  print(x,end=' ')
  f.close()
```

```
['Neha\n', 'Alisha\n', 'Amar\n', 'Aditya\n', 'Ashish\n']
Neha
 Alisha
 Amar
 Aditya
 Ashish
```

```
with open("myfile.txt",'w') as f:
  f.write("Python\n")
  f.write("is\n")
  f.write("easy\n")
  f.write("language\n")
  print("is file closed?",f.closed)

print("Now is file closed?",f.closed)
```

```
is file closed? False
Now is file closed? True
```

```
f=open("myfile.txt",'r')
print(f.tell())
print(f.read(2))
print(f.tell())
print(f.read(4))
```

```
0
Py
2
thon
```

```
data="Learing python id very easy from Ashish sir."
f=open("myfile.txt",'w')
f.write(data)

with open("myfile.txt",'r') as f:
  fileData=f.read()
  print(fileData)
  print("Current Cursor position is ",f.tell())
  f.seek(15)
  print("Current cursor position is ",f.tell())
  f.write("-JAVA-")
  f.seek(0)
```

```
  fileData=f.read()
  print(fileData)
```

```
Learing python id very easy from Ashish sir.
Current Cursor position is  44
Current cursor position is  15
---------------------------------------------------------------------------
UnsupportedOperation                      Traceback (most recent call last)
/tmp/ipykernel_526/3633410095.py in <cell line: 0>()
      9   f.seek(15)
     10   print("Current cursor position is ",f.tell())
---> 11   f.write("-JAVA-")
     12   f.seek(0)
     13   fileData=f.read()

UnsupportedOperation: not writable
```

```
import os ,sys
fname=input("Enter file name:")
if os.path.isfile(fname):
  print("File exist",fname)
  f=open(fname,'r')
else:
  print("File is not exist:",fname)
  sys.exist(0)

lcount=ccount=wcount=0
for line in f:
  lcount=lcount+1
  ccount=ccount+len(line)
  words=line.split()
  wcount=wcount+len(words)

print("Total lines:",lcount)
print("Total words:",wcount)
print("Total characters:",ccount)
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.